

ECE 2210: Python Programming for ECE

HW3 – Prompt user for input, Conditional Statements and Loops

The exercises should be done using only the statements and other features we have learned so far.

1. (20 pts) Complete the following functions in the given script **hw3.py**.
 - a) Complete the function named **rhombus()** that draws (i.e., prints) a rhombus like the following (Note that there is ALWAYS a whitespace after every asterisk), where the *height* of the rhombus is 5 (rows) and the symbol used to draw it is asterisk.

```
    *
  * * *
* * * * *
  * * *
    *
```

- b) Complete the function named “**draw_rhombus(h, s)**” that takes two parameters to draw a rhombus of height **h** using symbol **s** (e.g., ‘\$’, ‘%’, and ‘?’, etc.). It is assumed that **h** is always **odd**. You **MUST** use loop(s) to complete the drawing.
2. (20 pts) If any of the three lengths is **greater than** the sum of the other two, then you cannot form a triangle. Otherwise, you can. Complete the following functions in the given script **hw3.py**.
 - a) Complete the function named “**is_triangle()**” that takes three integers as arguments, and that prints either “**Yes**” or “**No**”, depending on whether you can or cannot form a triangle from sticks with the given lengths. **Hint**: Try to use one **if**-conditional statement with a compound condition.
 - b) Complete the function named “**interactive_is_triangle()**” that prompts the user to input the three stick lengths respectively, converts them to integers, and calls “**is_triangle()**” to check whether sticks with the given lengths can form a triangle. The three prompt messages are “**Enter l1>**”, “**Enter l2>**” and “**Enter l3>**” respectively. Always print the entered value right after each user’s input.
3. (15 pts) Complete the function named “**guess_number(k)**” to guess an integer between 1 and 9999 (both included). This predefined integer **k** is passed to the function as argument. The user is prompted to enter a guess (It is assumed the entered guess is a positive integer.).
 - a. If the user’s guess is out of the given range, user will get a “**Invalid input!**” message.
 - b. If user’s input is within the range but is wrong then the prompt appears again until the guess is correct.
 - c. On successful guess, user will get a “**Well guessed!**” message, and the program will exit. The prompt message is “**Enter number>**”.Always print the entered number right after the user’s input.
4. (25 pts) The mathematician Srinivasa Ramanujan found an infinite series that can be used to generate a numerical approximation of $1/\pi$:

ECE 2210: Python Programming for ECE

HW3 – Prompt user for input, Conditional Statements and Loops

$$\frac{1}{\pi} = \frac{2\sqrt{2}}{9801} \sum_{k=0}^{\infty} \frac{(4k)!(1103 + 26390k)}{(k!)^4 396^{4k}}$$

Complete the function called “**estimate_pi()**” that uses this formula to compute and return an estimate of π . It should use a **while**-loop to compute terms of the summation until the last term is smaller than $1e-15$ (which is Python notation for 10^{-15}).

5. (20 pts) The **divmod(x, y)** is a built-in function in Python that returns the quotient and remainder of **x** divided by **y**. For example,

```
>>> divmod(56, 3)
>>> (18, 2)
```

- Complete the function called **check_digit(n)** that returns **True** if all digits of positive integer **n** are even and returns **False** otherwise. **Hint:** The remainder (**== 3**) in the result of **divmod(123, 10)** gives the ones place value of 123. The quotient contains the rest of the places.
- Complete the function called **find_odd_digit_numbers(a, b)** that uses **check_digit(n)** to find all even-digit numbers between positive integer **a** and positive integer **b** (both included), and then print them.

Code testing:

The input is given in file “**input**”. The correct output is given in file “**output**”. Both “**input**” and “**output**” are text files. You can use any text editor (e.g., Sublime) to open them and view the contexts.

To test your code, put “**input**” and “**output**” in the same directory with “**hw3.py**”, and then run it in the Linux terminal using the following command, which generates a text file called “**myoutput**” that contains the output of your program.

```
$ python3 hw3.py > myoutput
```

Then, run the following command in Linux terminal to compare the output of your program and the given output.

```
$ diff output myoutput
```

If there is no output of running **diff**, it means the **output** and **myoutput** are identical. Note that TA will use the same command to test your program.

Hints: If **diff** shows difference exists between the two files, but you cannot see it with naked eyes, one possibility is that your **myoutput** file is generated on Windows while the **output** file I provided is produced on Linux. Since Windows and Linux use different control characters to represent line breaks, there’s always this invisible difference between Windows and Linux files containing multiple lines. Therefore, the only solution is to execute your code on Linux to generate the file that contains the output of your program.

Submission: For the submission of this assignment, you need to upload the completed **hw3.py** to Canvas.

Warning: The TA will compare your code with and your classmates’ codes using MOSS (<https://theory.stanford.edu/~aiken/moss/>). A similarity of 40% or higher is considered as

ECE 2210: Python Programming for ECE**HW3 – Prompt user for input, Conditional Statements and Loops**

plagiarism. The best way to avoid a high similarity is to write the code all by yourself. Discussion is always encouraged but never look at others' code.